

STL Rust Meetup

June 2026

Sponsors

Object Computing
Jetbrains
Manning



From Vibe to Verified Vibe less. Verify more.

Strong Specs • Iron Guardrails • Trusted Deliverable

<https://github.com/kmf-lab/from-vibe-to-verified>

Sponsor our STL Rust meetup:

<https://github.com/sponsors/kmf-lab>

The state of our industry

- 2026 Reality Check: 41–46% of all new Rust code is AI-generated
 - The nature of Rust is excellent for LLM generation
- Vibecoding is everywhere... and it's breaking our review pipelines
 - review time up 91%
- Code generation is 10× faster than review (the new bottleneck)

We need stronger guardrails

Specifications + Property Testing + Coverage + Mutations + Fuzz

Sources: Hostinger 2026 Vibe Coding Report, Faros AI 2025, GitHub 2025, Stack Overflow 2025

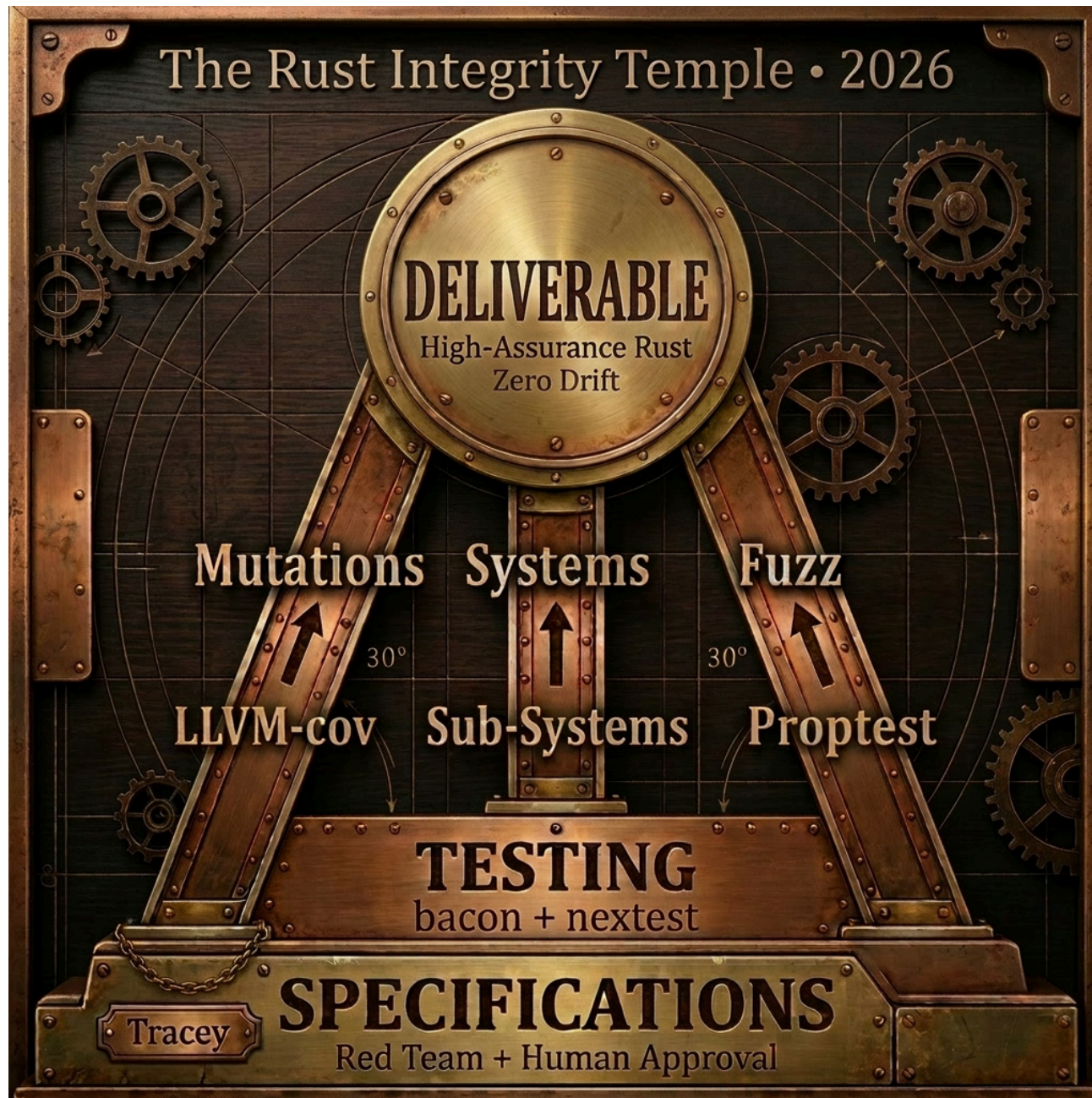
Rust Spec-Driven Testing Stack – 2026 Best Practices

Key Feature	test	nextest	<u>tracey</u>	llvm-cov	<u>proptest</u>	outdated	mutants	fuzz	bacon
Zero runtime production overhead	yes	yes	yes	yes	yes	yes	yes	yes	yes
Rich HTML / console reports	basic	yes	yes	yes	yes	yes	yes	yes	yes
Excellent CI/CD integration	yes	yes	yes	yes	yes	yes	yes	night	–
Fast checks as you work	detailed	yes	yes	yes	yes	manual	–	–	yes
Executes unit & integration tests	yes	yes	–	yes	yes	-	yes	–	yes
Executes documentation tests	yes	yes	–	yes	–	-	–	–	yes
Fast parallel execution + CI UX	basic	yes	–	-	–	-	–	–	–
Requirements / spec traceability	–	–	yes	–	–	-	–	–	–
Line+branch+region code coverage	–	–	–	yes	–	-	–	–	–
Property-based/generative-testing	–	–	–	–	yes	-	–	–	–
Outdated crate versions	–	–	–	–	–	yes	–	–	–
Mutation testing	–	–	–	–	–	-	yes	–	–
Coverage-guided millions fuzzing	–	–	–	–	–	-	–	yes	–
Checks panics, crashes & security	–	–	–	–	–	partial	Partial	yes	–
Automatic run on save (watch)	–	–	–	–	–	-	–	–	yes

I am still on this journey and there is a lot more to try out.

Honorable mentions for future study: Miri, Shuttle, Contract, Hack

Proverbs 10:9 “He who walks in integrity walks securely, But he who makes his ways crooked will be found out.”



Strong Specs • Iron Guardrails • Trusted Deliverable

101 Basic Rust Testing

- cargo test (The Starting Point)
 - Tests are marked with `#[test]`
 - Tests can return `Result<(), Box<dyn std::error::Error>>`
 - Other attributes: `#[should_panic]`, `#[ignore]`
- Where tests live
 - Unit tests: inside your code using `#[cfg(test)] mod <SOME NAME>_test {`
 - Completely stripped from release binaries, never shipped
 - Has private access
 - Integration tests: are in the `tests/` folder
 - Does not have private access
 - Documentation tests: are in the `///` comment blocks
 - Optional: Does not always work well with the IDEs
 - Run with `cargo test -doc`



Example Project: stockviz

sealed_test: Isolated Test Environments

- Runs every test in its own isolated subprocess + fresh temporary directory
- Environment variables are safely set inside the `#[sealed_test]` annotation
 - no `unsafe set_var` needed
- Completely eliminates flaky tests caused by shared env state
- Zero runtime overhead, simple drop-in replacement for `#[test]`

Update your Cargo.toml

```
[dev-dependencies]
sealed_test = "1.1"
```


sealed_test: Example

```
use sealed_test::prelude::*;

#[sealed_test(env = [ ("API_KEY", "test-secret-123"),
                      ("DEBUG", "true") ])]

fn test_with_isolated_env_and_files() {
    // Env vars set via the annotation are isolated to this test only
    assert_eq!(std::env::var("API_KEY").unwrap(), "test-secret-123");
    assert_eq!(std::env::var("DEBUG").unwrap(), "true");

    // Files are also isolated in a clean temp directory
    std::fs::write("data.txt", "hello from sealed_test").unwrap();
    assert_eq!(
        std::fs::read_to_string("data.txt").unwrap(),
        "hello from sealed_test"
    );
}
```


nextest: The Next-Generation Test Runner

What's wrong with “plain cargo test?”

It's good... but slow on large codebases, basic reporting, and doesn't scale

- Much faster parallel execution (each test in its own process)
- Excellent CI/CD integration & Rich HTML console reports
- Built-in flaky-test detection & retry, off by default

◦ `cargo nextest run --retries 2`



- Runs unit + integration tests efficiently
- Still supports `cargo test --doc`

This is the speed layer that lets us keep up with the large volume of AI-generated code.

install: `cargo install cargo-nextest --locked`

Use `locked` to ensure we use the dependencies the author shipped with!!

llvm-cov: See What Your Tests Actually Exercise

- Line coverage, Branch coverage, Region coverage (most precise)
 - Beautiful, actionable reports
 - Zero runtime overhead
 - Perfect for CI quality gates
- Clean HTML dashboards with color highlighting 
 - `cargo llvm-cov nextest --html --open`
- Console summaries for **quick checks and AI handoff**
 - `cargo llvm-cov nextest --show-missing-lines`
- Run llvm-cov when you see bacon + nextest are happy
 - Instant visibility into which parts of your code are covered.

install: `cargo install cargo-llvm-cov --locked`

Key limitation: Coverage shows how much code ran. It does not show whether the right things are being tested or if requirements are met; that's why we pair it with Tracey (and later proptest + mutants).

bacon: Tests That Run While You Code

Bacon watches your files and automatically re-runs the relevant tests in the background the moment you save.

- Runs unit tests, integration tests, and documentation tests
- Rich, clean console output (no noise and instant feedback)
- Non-blocking, stays out of your way
- Works with cargo test, nextest, and more
 - Use ctrl-j to switch between jobs
- Zero configuration for most projects
 - `bacon nextest`

Just open a terminal and run `bacon` (`bacon test` / `bacon nextest`).
Keep coding. Tests update live. Fast checks as you work.

install: `cargo install --locked bacon --features "clipboard"`

Issue: Clipboard may not work on pop_os!

```
sudo apt install xclip  
sudo apt install wl-clipboard
```

bacon + nextest + llvm-cov

Setup copy to clipboard:

```
bacon -prefs
```

Edit: `~/.config/bacon/prefs.toml` and insert into the `[keybindings]`

```
ctrl-y = "copy-unstyled-output"
```

Setup our own coverage job:

```
bacon -init
```

Add our own custom job to `bacon.toml`:

```
[jobs.coverage]
command = [
    "cargo",
    "llvm-cov",
    "nextest",
    "--color",
    "always",          # keeps nice colored output in Bacon
]
need_stdout = true
analyzer = "nextest" # built-in support for nextest output parsing
```

Then we launch continuous watching coverage:

```
bacon coverage
```



tracey: Your traceability engine for Rust

This is the central glue of our AI test stack and the one thing you need to take home

One goal: Prevent spec drift before it breaks anything.

- Code changes fast, and even faster now with AI
 - Tests drift away from original intent found in **old docs**
- Works alongside bacon, llvm-cov, nextest, proptest, etc.

Tracey automatically links:

- Requirements (in Markdown specs) with Tests (in Source)
- Implementation code with Dependent and Related code

Tracey turns your documentation into a living, enforceable contract for humans and LLM agents. **Know what is implemented and tested.**

How to use tracey

Install:

The first release was Dec 2025 !! This is the new hotness !!

```
npm install -g pnpm  
cargo install --locked --git https://github.com/bearcove/tracey --branch main tracey
```

Super lightweight, no heavy framework, zero runtime overhead:

1. Setup paths in the project: `.config/tracey/config.styx`
2. Define requirements in Markdown using `r[req.id]`
3. Add tiny annotations in your Rust code comments

Requirements in Markdown:

- Just `r[requirement.id]` tags.
- Versioning (+2) automatically catches stale references.

```
## Authentication, simple spec file (e.g. docs/spec/auth.md)
```

```
r[auth.login]
```

```
Users MUST log in with valid credentials.
```

```
r[auth.rate-limit+2]
```

```
Failed attempts MUST be rate-limited (versioned).
```

tracey: Linking Code & Tests to your Requirements

Verbs needed at the front of each anchor comment:

- Impl - The implementation of this requirement in your source
- Depends - Something in your source which depends upon this requirement
- Related - Something which may be good to know perhaps a helper method
- Verify - A test which will verify your implementation

Rust source example; just comments. No macros and No boilerplate

```
// r[impl auth.login]
pub fn login_user(...) -> Result<...> { ... }

// r[impl auth.rate-limit+2]
pub fn limit_rate(...) -> Result<...> { ... }

// r[depends auth.rate-limit+2]
pub fn user_audit(...) -> Result<...> { ... }

#[cfg(test)]
mod tests {
    // r[verify auth.login]
    #[test]
    fn login_succeeds_with_valid_creds() { ... }
}
```


tracey Integrations

View the full traceability matrix: `tracey web --open` 

- Instant browser launch
 - starts the interactive web dashboard and automatically opens it in your default browser
- **Spec View**
 - rendered requirements with live status badges + one-click links straight to implementations/tests.
- **Coverage View**
 - filterable table of every requirement showing impl/verify coverage (perfect for spotting gaps fast).
- **Sources View**
 - file tree with coverage badges + syntax-highlighted code displaying all your `r[impl]`, `r[verify]`, etc.
- **Global search**
 - Cmd/Ctrl+K to search across all requirements instantly.
- **Real-time & visual**
 - turns your annotations into an always-up-to-date, clickable traceability browser

Add hotlink into RustRover with LSP:

Install plugin **LSP4IJ** (Language Server Plugin 4 IntelliJ) 

Then add the language server “Tracey LSP” which should run “`tracey lsp`”
Set the mappings for this to rust and markdown

Quick CLI commands: `tracey query <subcommand>`

<code>status</code>	coverage overview // you may want to add this to <code>bacon.toml</code> as a job
<code>uncovered</code>	List rules without implementation references
<code>untested</code>	List rules without verification references
<code>unmapped</code>	Show unmapped code units
<code>stale</code>	List stale references (code pointing to older rule versions)
<code>rule</code>	Show details about one or more rules
<code>config</code>	Display current configuration
<code>validate</code>	Validate the spec and implementation



MCP Integration:

```
"mcpServers": {  
  "tracey": {  
    "command": "tracey",  
    "args": ["mcp"],  
    "stdio": true,  
    "enabled": true  
  }  
}
```

Add this skill to your editor:



```
curl -o ~/.cursor/skills/tracey/SKILL.md
```

<https://raw.githubusercontent.com/bearcove/tracey/main/crates/tracey/skill/SKILL.md>

Tracey: Central Glue for the Stack; Why it matters

tracey web becomes your spec health check. Now your entire testing stack is intelligent and drift-proof.

- Keeps LLMs/agents aligned with your actual requirements
 - Prevent drift and keep the LLM in check
 - The spec is the source of truth as inputs not afterthoughts
- Turns “vibe coding” into **spec-driven engineering**
- Highly recommend a “red team” of your spec document before development begins
- Keeps these together as we work: bacon (fast feedback), nextest (speed), proptest/mutants/fuzz (quality), tracey (traceability)
 - Run `tracey web` AND `bacon coverage`

The podcast kicking it off: <https://sdr-podcast.com/episodes/traceability/>

<https://tracey.bearcove.eu/>



<https://github.com/bearcove/tracey>

<https://crates.io/crates/tracey>

<https://docs.rs/crate/tracey>

proptest: Property-Based Testing for Rust

QuickCheck (Haskell 1999) - Hypothesis (Python 2013) - Proptest (Rust 2017)

Example-Based vs Property-Based

- Traditional tests (what we've used so far)
 - You write specific inputs and expected outputs.
 - Example `Sort([3,1,2]) == [1,2,3]`
- Proptest (property-based)
 - You write “for all possible inputs, this property must hold”
 - Example `true == Sort(some_vec).windows(2).all(|w| w[0] <= w[1])`
 - Far more powerful than example-based tests

Define properties that must always be true; then let the tool generate thousands of random inputs automatically.

With lots of LLM-generated code why is proptest essential?

- LLMs are great at happy paths but weak on edge cases
 - Vibe coding creates subtle bugs that only appear in production
- Using thousands of random cases it finds many hidden bugs
 - Manual tests miss weird inputs you will **not** think of
- Providing clear tiny repeatable failure examples
 - Clear test case so the LLM knows when it has been fixed.

proptest automatically finds corner cases for you

1. Define Property

- You write invariants your code **must always satisfy**.
- Use `proptest!` macro **or** the `#[proptest]` attribute + `prop_assert!` macros.

2. Specify Strategies

- Tell proptest **how** to generate smart random inputs.
- Use built-ins like `any : <T> ()`
- Or define custom Strategy for complex types

3. Generate Random Cases

- Proptest automatically creates **hundreds of test cases** (default = 256).
- It uses a random number generator + ValueTree to produce varied inputs.

4. Execute Tests

- Runs your property function on **every** generated input.

5. On Failure; Shrink! (**Killer Feature**)

- When a test fails, proptest **automatically shrinks** the failing input to the **smallest possible counterexample**. Uses the ValueTree above.
- This turns huge random failures into tiny, easy-to-debug cases.

6. Report & Replay

- Prints the minimal failing case (You should see this in bacon coverage) 
- **Saves the random seed** in `proptest-regressions/` folder
- Next run replays the exact same failure for perfect reproducibility

proptest example: custom vec

```
use proptest::prelude::*;

// 1. Custom Vec (the #1 most common custom input)
proptest! {
    //by default only 256 cases are generated but you should customize this.
    #![proptest_config(ProptestConfig::with_cases(1000))] // or any number > 256
    #[test]
    fn sorting_is_idempotent(
        mut v in prop::collection::vec(any::<i32>(), 0..100) // 0 to 99 of any i32
    ) {
        let mut sorted_once = v.clone();
        sorted_once.sort();
        let mut sorted_twice = sorted_once.clone();
        sorted_twice.sort();
        prop_assert_eq!(sorted_once, sorted_twice);
    }
}
```

Avoid writing dozens of hand-crafted examples.

proptest example: custom struct

```
use proptest::prelude::*;

// 2. Custom struct via #[derive(Arbitrary)] - also used by fuzz later
#[derive(Debug, Clone, PartialEq, Arbitrary)]
struct Point {
    x: i32,
    y: i32,
}

proptest! {
    #[test]
    fn point_roundtrip(p: Point) {
        prop_assert_eq!(p, p); // trivial property - just shows it generates
    }
}
```

Instead of writing individual test cases, you define properties (invariants) that must always hold, then let the library generate thousands of random inputs to try to break them.

proptest example: filtered custom input

```
use proptest::prelude::*;
// 3. Filtered custom input (.prop_filter)
proptest! {
    #[test]
    fn only_positive(
        n in (0..100i32).prop_filter("must be positive", |&x| x > 0)
    ) {
        prop_assert!(n > 0);
    }
}
```

Works out of the box with bacon (instant feedback) and nextest (fast parallel runs).
No new CI changes needed only these dependencies.

```
[dev-dependencies]
proptest = "1.11"
proptest-derive = "0.8"
```


proptest example: custom composed strategy

```
use proptest::prelude::*;

// 4. Custom composed strategy with prop_compose! (most powerful)
prop_compose! {
    fn even_pair() (a in 0..50i32, b in 0..50i32) -> (i32, i32) {
        (a * 2, b * 2)
    }
}

proptest! {
    #[test]
    fn both_even((a, b) in even_pair()) {
        prop_assert_eq!(a % 2, 0);
        prop_assert_eq!(b % 2, 0);
    }
}
```

<https://github.com/proptest-rs/proptest>

<https://proptest-rs.github.io/proptest>



outdated: Keep your crates fresh

One outdated crate = one potential supply-chain attack

This is your “first line of defense” in the testing + security stack

Compares every crate in your dependency tree against the latest published versions on crates.io

Shows:

- semver-compatible updates
- major version bumps
- security-critical updates

Supports workspace projects and custom ignore lists.

Fail the build if any critical or high-severity updates are available.

Might want to add a bacon job for outdated but CI is probably a better place.

Commands:



```
cargo outdated --exit-code 1 // quick check you should run daily AND add to your CI pipeline
cargo update                // easy update to the latest compatible crates
```

mutants: test quality layer

Measure how strong your tests really are.

We inject small bugs into your code and check if your tests catch them.

cargo-mutants does three simple things:

1. Creates hundreds of tiny “mutants” (e.g. flips `>` to `<`, changes `true` to `false`, removes a check ...)
2. Runs your full test suite against each mutant
3. Given a Mutant Details
 - a. A mutant is killed if any test fails: good!
 - b. A mutant survives if all tests still pass: weak spot!

Mutation score = % of mutants killed.

Higher score = stronger tests.

Typical output:



```
MISSED src/chart.rs:81:20: replace || with && in layout_column_count in 3s build + 1s test
MISSED src/chart.rs:106:42: replace - with + in last_bar_center_x in 3s build + 2s test
MISSED src/chart.rs:106:42: replace - with / in last_bar_center_x in 3s build + 2s test
MISSED src/chart.rs:164:5: replace debug_assert_sma_candle_x_alignment with () in 3s build + 2s test
MISSED src/chart.rs:164:20: replace || with && in debug_assert_sma_candle_x_alignment in 3s build + 1s test
MISSED src/chart.rs:164:15: replace == with != in debug_assert_sma_candle_x_alignment in 3s build + 1s test
```

mutants: Valid Mutations

cargo-mutants applies these 6 mutation patterns:

- Replace entire function body
with a type-appropriate dummy value
(e.g. `false` $\Leftrightarrow$ `true`, `0`, `None`, `vec![]`, `String::new()`, `Default::default()`, `Ok(x)` $\Leftrightarrow$ `Err(...)`)
- Flip binary operators
 - `>` $\Leftrightarrow$ `<` OR `==`
 - `==` $\Leftrightarrow$ `!=`
 - `&&` $\Leftrightarrow$ `||`
 - `+` $\Leftrightarrow$ `-` OR `*`
 - `/` $\Leftrightarrow$ `%`
 - etc
- Remove unary operators (delete `!` or unary `-` entirely)
- Delete match arm (only when a `_` wildcard arm exists)
- Replace match guard (swap guard expression to `true` or `false`)
- Delete field from struct literal, when `..Default::default()` or `..base` is present

mutants: our test quality layer

Detailed documentation: <https://mutants.rs> 

Install: `cargo install cargo-mutants --locked`

Configuration: `.cargo/mutants.toml`

```
exclude_globs = ["src/gui/**", "src/main.rs", "fuzz/**"]
```

RUN: `cargo mutants`

Mutants often take days or weeks at a time !

- Could be configured to run on a single core

- Mutants provides a diff feature for smaller runs

- Consider using Ramdisks for the Rust build process

- Consider the faster mold & Wild linkers for Rust

- Consider cargo profiler changes for speed over build time

- Spin up an old machine in the basement ;-)

fuzz: Coverage-Guided Fuzzing

robustness & security layer in the Rust testing stack

It throws millions of random inputs at your code automatically, **hunting** for panics, crashes, memory errors, and security vulnerabilities that no human (or LLM) would ever think to test.

Leverages the same `#[derive(Arbitrary)]` we already added for proptest usage.

<https://rust-fuzz.github.io/book/cargo-fuzz.html>

<https://github.com/rust-fuzz/cargo-fuzz>

Fuzzing is unlike normal tests or proptest:

- cargo-fuzz generates billions of inputs intelligently
- Uses coverage guidance (from llvm-cov) to explore new code paths
- Looks for anything that causes a panic, crash, or undefined behavior

When it finds a bug, it gives you the exact minimal input that triggers it; instantly reproducible.

Works great in CI and can be left running longer on nightly/release prep.

fuzz: Usage

It's your automated “chaos engineer”; catching panics, buffer issues, and security vulnerabilities before they ship

Install:

```
cargo install cargo-fuzz --locked
rustup install nightly
cargo +nightly fuzz init //works only once to create the fuzz_targets folder
```

Usage:



```
cargo +nightly fuzz add my_target # creates empty fuzz/fuzz_targets/my_target.rs
cargo +nightly fuzz run my_target -- -max_total_time=300 # 5-minute run
cargo +nightly fuzz cmin my_target # minify your input files (for faster runs)
```

```
cargo +nightly fuzz coverage my_target ## report the coverage of the fuzz test
```

New profdata is only compatible with the nightly llvm-cov ...

```
llvm_cov="$ (rustc +nightly --print sysroot)/lib/rustlib/$ (rustc -vV | grep '^host:' |
cut -d' ' -f2)/bin/llvm-cov"
```

```
$llvm_cov report --instr-profile=fuzz/coverage/my_target/coverage.profdata
target/x86_64-unknown-linux-gnu/coverage/x86_64-unknown-linux-gnu/release/my_target
--sources src/ --ignore-filename-regex='/.cargo/registry' --use-color
```

Short skill to populate “my_target”

~/.cursor/skills/rust-fuzz-target-expert/SKILL.md



Workflow: Vibe less, Verify more

Daily as you develop and explore:

bacon coverage #custom job with nextest, proptest, llvm-cov
tracey web –open

Periodicly as you do PR/CICD:

cargo outdated
cargo mutations
cargo +nightly fuzz run <target>

This combination gives you the full spectrum: fast local feedback (bacon), correctness (proptest), visibility (llvm-cov + tracey), robustness (fuzz), test quality (mutants), and speed (nextest). Spec traceability is enforced on every change

Expect these guard rails to lower your token costs!!

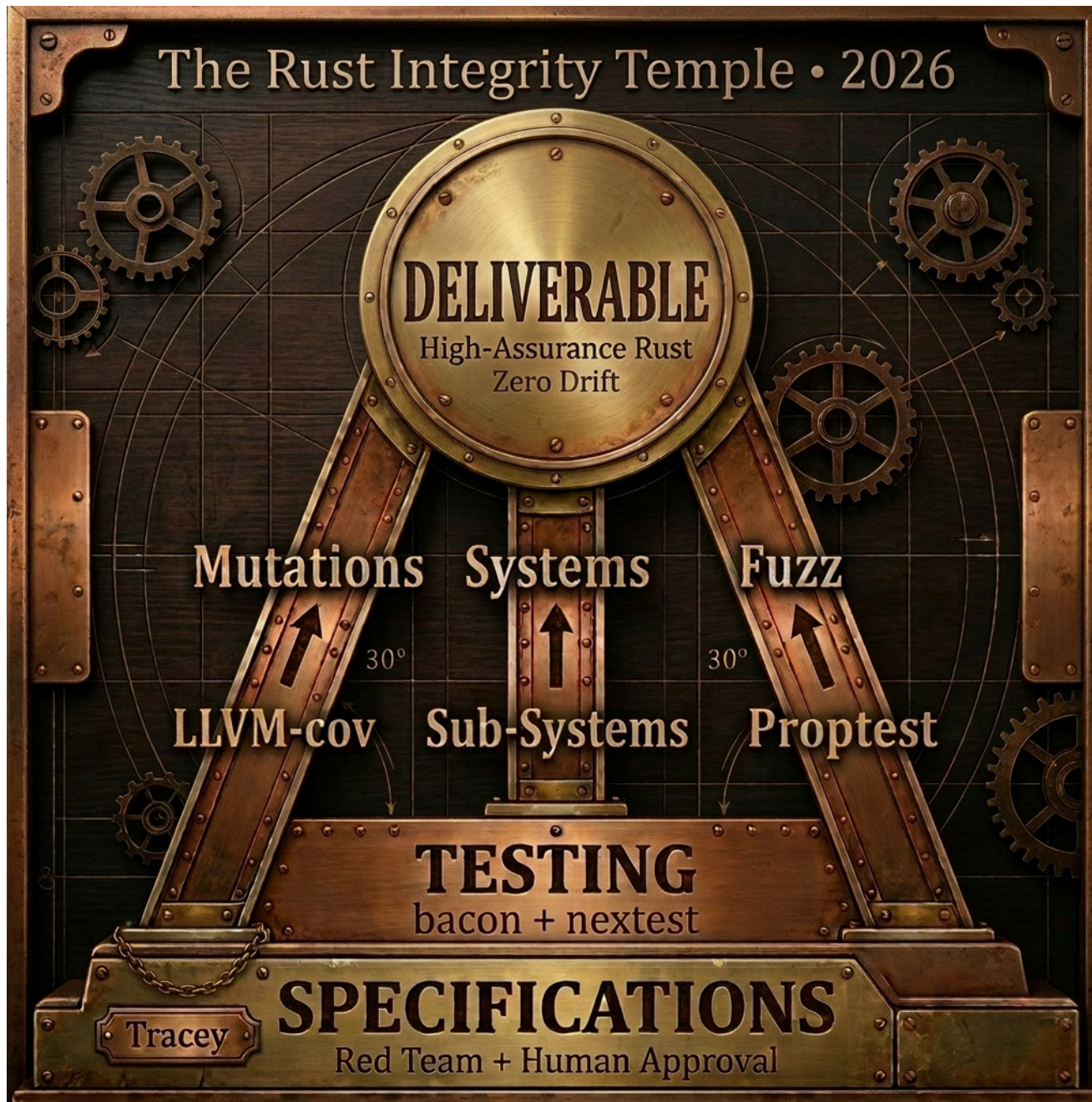
- Tracey specification links keep LLM focused
- Imagine how many tokens you would burn to reach this level of quality without these tools!!

What are we missing?

Counterproductive: clippy, rustfmt, insta ...

Next step investigations: miri, shuttle, contracts, hack ...

Help sponsor our meetup: <https://github.com/sponsors/kmf-lab>



Testing Quiz

1. Basic Test Attributes & `cfg(test)`

Question: Which of the following is true about this code?

```
#[cfg(test)]
mod tests {
    #[test]
    fn it_works() {
        assert_eq!(2 + 2, 4);
    }
}
```

- A) The test only runs when you do `cargo test --lib`
- B) The test is compiled into the final binary
- C) The test module is completely stripped in release builds
- D) The test will fail to compile because `main` is missing

Correct: C Explanation: `#[cfg(test)]` means the entire module is only compiled when running tests (cargo test). It is never included in cargo build or release binaries. This is why you can put helper functions and even private items inside the test mod without polluting your crate.

2. `#[should_panic]` vs Expected Message

Question: Does this test pass?

```
#[test]
#[should_panic(expected = "division by zero")]
fn test_divide() {
    let _ = 10 / 0;
}
```

- A) Yes — it panics with the right message
- B) No — `expected` only works with `panic!` macro, not integer division
- C) Yes — Rust normalizes all panic messages
- D) Compilation error — `expected` attribute is invalid syntax

Correct: B Explanation: Integer division by zero panics with the message "attempted to divide by zero", not "division by zero". The expected parameter does exact substring matching, so this test fails. (Classic gotcha that always sparks discussion.)

3. Returning Result from Tests

Question: What happens when you run this test?

```
#[test]
fn test_file() -> Result<(), Box<dyn std::error::Error>> {
    let _ = std::fs::File::open("non_existent.txt");
    Ok(())
}
```

- A) The test always passes
- B) The test fails with a nice error message showing the original `io::Error`
- C) Compilation error — tests cannot return `Result`
- D) The test panics instead of returning `Err`

Correct: B Explanation: Since Rust 1.0, tests can return `Result<T, E>` where `E: Debug`. If the test returns `Err`, Cargo prints the error nicely and marks the test as failed. This is the modern, idiomatic way to test fallible code.

4. Testing Private Functions

Question: Can this test compile and access `double`?

```
fn double(n: i32) -> i32 { n * 2 } // private
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_double() {
        assert_eq!(double(5), 10);
    }
}
```

- A) No — private items cannot be tested
- B) Yes — tests inside the same crate/module can access private items
- C) Yes, but only if you add `pub(crate)`
- D) No — you must move the test into an integration test

Correct: B Explanation: Unit tests (in the same crate) have full access to private items. This is one of the biggest advantages of Rust's unit-test model over languages that require everything to be public for testing.

5. `#[ignore]` + `cargo test --ignored`

Question: What command runs only this test (and other ignored ones)?

```
#[test]
#[ignore = "expensive database test"]
fn test_db_integration() { /* ... */ }
```

- A) `cargo test test_db_integration`
- B) `cargo test --ignored`
- C) `cargo test -- --ignored`
- D) `cargo test --test-threads=1`

Correct: B Explanation: cargo test --ignored is the dedicated flag. It's extremely useful in CI or when you want to run slow/expensive tests separately. The = "reason" part is just documentation (shown in cargo test output).

6. Documentation Tests (/// examples)

Question: This doc test will...?

```
/// ```  
/// let x = 5;  
/// assert_eq!(x, 6); // deliberate failure  
/// ```  
pub fn my_function() {}
```

- A) It is ignored because it's not marked `no_run`
- B) It fails when you run `cargo test` (and also `cargo test --doc`)
- C) It only runs with `cargo test --doc` and passes silently
- D) Compilation error — doc tests cannot contain assertions

Correct: B Explanation: Any code block in `///` or `///!` docs is automatically compiled and run as a test when you do cargo test. This is why documentation examples stay correct forever. (The failure here is intentional to show the point.)

7. Interior Mutability in Tests (tie-in to your previous set)

Question: You want to test a function that needs to mutate shared state. Which is the most idiomatic in a unit test?

```
#[test]
fn test_counter() {
    let counter = std::cell::Cell::new(0);           // Option A
    // or
    let counter = std::cell::RefCell::new(0);        // Option B
    // or
    let counter = std::sync::Mutex::new(0);          // Option C
    // ... use counter ...
}
```

- A) Cell — fastest for Copy types
- B) RefCell — perfect for single-threaded tests
- C) Mutex — safest, works even if test becomes async later
- D) None — you should always use `&mut` instead

Correct: B Explanation: RefCell is the sweet spot for unit tests: zero runtime cost in release, clear panic messages on borrow errors, and single-threaded by nature. Cell only works for Copy types. Mutex adds unnecessary overhead and PoisonError boilerplate in pure unit tests.